

Patch-Aware Knowledge Graph Retrieval for LLM-Based Repository-Level Test Generation

Joesiah Liu, Miguel Varilla, Sheryl Lee, and Thong Wan Mun

Abstract—While Large Language Models (LLMs) show promise for repository-level automated test generation, a substantial portion of their generated tests are invalid due to poor context, hallucinations, and unresolved dependencies. Existing RAG methods struggle to isolate the change-centric relationships needed for accurate test oracles, which specify the expected outcome used to determine whether a generated test passes or fails. To address this limitation, we propose a patch-aware KG retrieval framework for LLM-based test generation, which constructs a repository KG modelling structural dependencies and extracts change-anchored subgraphs to provide the LLM with context specific for test oracle generation. Evaluated against no-retrieval and text-RAG baselines, our approach aims to improve test correctness, coverage, and mutation scores by mitigating hallucinations and refining oracle generation.

I. INTRODUCTION

LLM-based coding tools like GitHub Copilot, Cursor, and Claude Code have transformed repository-level software engineering, particularly automated test generation, which addresses the high cost of manual test maintenance [1], [2]. However, the practical value of these tools depends entirely on output quality: generated tests must be syntactically valid, compilable, and behaviourally correct. Consequently, improving the reliability and contextual grounding of repository-level LLM test generation has become a critical research challenge.

A. Background and Motivation

Test generation is more complex than code generation. Where code generation produces implementation logic, test generation requires implicit understanding of intended behaviour to construct accurate assertions and test oracles, which acts as a source of truth for the generated tests [3].

Although LLMs excel at self-contained tasks, repository-level performance degrades due to untargeted retrieval of irrelevant code, a context-change mismatch that misses modification boundaries, and an absence of semantic grounding to anchor context to specific changes. [4], [5]. Consequently, LLM hallucinations such as non-existent API calls, unresolved dependencies, and inconsistent cross-file logic can cause generated tests to fail compilation, induce false positives, or diverge from intended behaviour [6].

At the core of these limitations lies the test oracle problem: current methodologies predominantly ensure syntactic validity while failing to guarantee behavioural correctness, historically

necessitating expensive human review to verify developer intent [7].

B. Related Works and Gaps

Although Retrieval-Augmented Generation (RAG) reduces hallucinations by supplying LLMs with relevant code snippets [8], current mechanisms lack anchors for localised code changes. This results in broad, noisy context. While RAG improves syntactic and API-level grounding by resurfacing existing code, it offers limited support for patch-specific behavioural expectations, leaving the test oracle problem unresolved.

To capture structural context more effectively, KGs have been used to model relationships such as function calls, imports, class hierarchies, and API usage [9]. However, many KG approaches rely on static, full-repository context rather than localised code deltas, which introduces irrelevant context and computational overhead in dynamic test generation. Moreover, structural topology alone does not resolve the oracle problem, as graph-based representations do not directly infer developer intent or expected behaviour.

To bridge rigid structure and behavioral inference, we leverage neuro-symbolic integration [10]. Specifically, we adopt a symbolic $\rightarrow$ neural approach, injecting structured symbolic constraints into the LLM input to ground and bound its stochastic generation behavior.

Rather than relying on the LLM to construct or update the KG, we feed pre-validated symbolic context directly to the model. Because this context is verified prior to inference, it grounds test generation in deterministic repository facts rather than stochastic model assumptions, preventing hallucinated dependencies. As summarized in Table I, no existing approach combines KGs, RAG, and LLMs specifically for patch-aware test generation.

TABLE I
COVERAGE OF RELATED APPROACHES ACROSS KEY DIMENSIONS

Method	KG	LLM	RAG	Test/Code Gen
GraphEval [11]	✓	✓	—	—
SemanticForge [9]	✓	✓	—	Code
GraphCoder [12]	✓	✓	✓	Code
CGM [13]	✓	✓	✓	Code
TESTGENEVAL [3]	—	✓	—	Test
SWT-Bench [14]	—	✓	—	Test
Ours	✓	✓	✓	Test

J. Liu and W.M. Thong are with Monash University, Clayton, VIC 3800, Australia. (e-mail: jliu0290, wtho0016@monash.edu). M. Varilla and S. Lee are with Monash University Malaysia, Subang Jaya, Selangor, Malaysia (e-mail: mvar0010, wlee0060@monash.edu).

While prior work combines some of these components, existing retrieval and KG-based methods remain weakly connected to active code modifications and are often optimised for

general code generation rather than repository-level test synthesis, where tests are generated using the broader repository context, including related files, dependencies, and code structure. This motivates a patch-aware KG retrieval framework that extracts validated, change-centric subgraphs for LLM-based test generation.

C. Problem Statement and Research Questions

Current LLM-based test generation suffers from a lack of change-aware, localized retrieval. Without sufficient semantic grounding to infer expected behavioral changes, models produce invalid or misleading test suites with faulty or absent oracles. This stems from three core architectural limitations:

- **Untargeted Retrieval:** Broad, full-repository context introduces irrelevant code that degrades model performance.
- **Context-Change Mismatch:** Static context fails to capture the precise boundaries and semantic implications of code modifications.
- **Absence of Semantic Grounding:** Existing retrieval mechanisms do not explicitly anchor context to specific changes, failing to support accurate oracle generation.

Repository-level test generation therefore requires localised, change-aware context that is explicitly grounded in the patch signal. To evaluate the feasibility and effectiveness of this direction, this study addresses four research questions:

- **RQ1 (KG Construction Quality):** How accurately does the repository-level KG capture code structure, dependencies, and API relationships?
- **RQ2 (Patch-Based Subgraph Retrieval):** How effectively can code patches and issue reports identify affected entities and retrieve relevant KG subgraphs?
- **RQ3 (Test Generation Correctness and Coverage):** To what extent does patch-aware KG subgraph retrieval improve the correctness and coverage of LLM-generated test cases compared to retrieval-free baselines?
- **RQ4 (Efficiency):** What is the computational cost of the proposed approach relative to the quality gains achieved?

D. Aim and Objectives

Driven by these research questions, this research aims to develop a patch-aware KG retrieval framework that improves the correctness, coverage, and oracle accuracy of repository-level LLM-generated tests. This aim is addressed through four technical objectives:

- 1) **Repository-Level Knowledge Graph Construction** (*Sem. 1, Weeks 8–10*): Construct a repository-level KG capturing structural, dependency, and API relationships via Abstract Syntax Trees (AST).
- 2) **Change-Aware Subgraph Extraction** (*Sem. 2, Weeks 1–4*): Design and validate an extraction mechanism anchored to patch-affected entities utilising bounded Breadth-First Search (BFS) expansion.
- 3) **Neuro-Symbolic Prompt Integration** (*Sem. 2, Weeks 1–4*): Integrate structurally validated subgraphs as structured context into language model prompts to tightly constrain test and oracle generation.

- 4) **Empirical Framework Evaluation** (*Sem. 2, Weeks 5–8*): Evaluate the framework against language-model-only and untargeted retrieval baselines across four core metrics: test correctness, code coverage and mutation score.

II. METHODOLOGY

As shown in Fig. 1, our methodology constructs a repository-wide KG, extracts a focused subgraph around patch-modified functions, and formats this localised structure as context for LLM test generation. Generated tests are subsequently executed to evaluate coverage and correctness.

A. Knowledge Graph Construction

To build the KG, repositories are cloned as bare mirrors. Construction proceeds in two distinct phases: Parallel Extraction and Sequential Resolution.

a) Parallel Extraction: Each .py file is parsed into an AST. Abstract nodes (classes, methods, functions, imports) are mapped to graph nodes, while abstract relations (inheritance, instantiations, function calls) are emitted as directed edges.

b) Sequential Resolution: Extracted nodes are aggregated into a global index. Unresolved call targets are matched against this index and tagged by confidence: *exact* (single match), *ambiguous* (multiple matches), or *dropped* (external libraries). Call contexts (caller counts and direct callers) are then attached to each node. The resulting deterministic graph uses MD5-hashed component names for reproducibility and is persisted as JSON.

B. Patch Parsing and Seed Identification

Given a unified patch and a target file, the parser identifies modified functions and classes by:

- 1) Parsing the unified diff into per-file change hunks.
- 2) Locating hunks within the target code file.
- 3) Extracting changed function or class definitions via regex pattern matching.
- 4) Returning a set of modified entity names to serve as seed nodes.

These changes are then queried against the KG to locate corresponding seed nodes and represent the modified functions that need test coverage. If a test file exists in the repository, it is also added as a seed to enable test discovery.

C. Subgraph Extraction

From seed nodes, a Breadth-First Search (BFS) extracts surrounding context up to a configurable depth which has shown to be effective in capturing immediate dependencies within a repository [15]. The algorithm traverses both outgoing edges (function calls, inheritance, instantiation) and incoming edges (callers, subclasses), collecting related nodes and edges.

Edge filtering excludes low-signal edges (e.g., imports, {module_depends_on}) which scatter context across unrelated code. This is consistent with established practice in graph-based subgraph extraction [16]. The extracted sub graph includes:

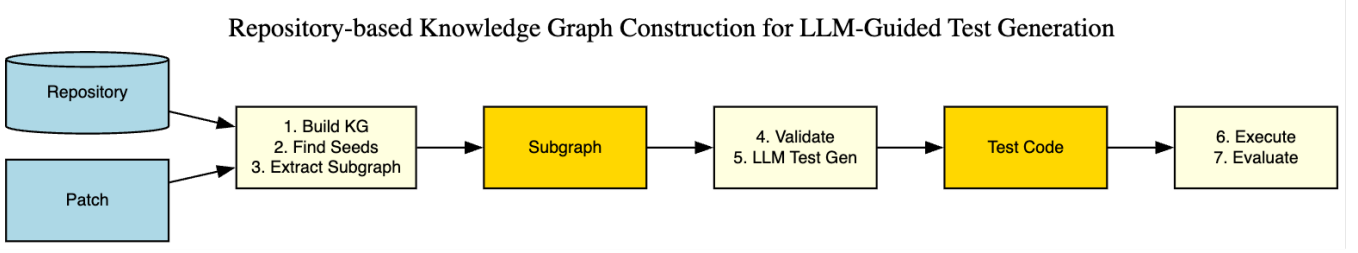


Fig. 1. The overview of patch-aware KG retrieval for repository-level test generation.

- **Seed nodes:** the modified functions
- **Context Nodes:** related functions (callers and call-ees), parent/child classes, test functions
- **Edges:** relationship within the sub graph

D. Subgraph Validation

The extracted subgraph is validated to ensure it does not contain the following structural violations:

a) *Pruning Isolated Nodes:* Isolated nodes in the KG provide no relational context, breaking link-based reasoning. Including these orphaned nodes adds noise without signal, as the LLM cannot reason about their callers, dependencies, or call-ees.

b) *No Broken Edges:* Dangling edges create logical inconsistencies. If function B calls function A, but A is not in the subgraph, the LLM cannot reason about A's behaviour, leading to incomplete or incorrect test generation.

c) *Seeds Connected via Subgraph Edges:* The seed represents the code entity under test. If it has no edges connecting it to the rest of the subgraph, the LLM receives no context about how the code integrates with callers, dependencies, or existing tests which severely degrades test quality. Seeds must maintain at least one edge connection to the broader subgraph.

d) *No Duplicate Edges:* Duplicate edges add no information and only consume tokens in the LLM's limited context window, reducing the effective density of useful context. They inflate edge counts used in metrics and debugging.

E. Neuro-Symbolic Integration

For efficient LLM consumption, the validated subgraph is reformatted into a hierarchical JSON structure organised by semantic relationships. The hierarchical format organises information into three semantic sections:

- **Seed Section:** Contains the modified function(s) at the centre of analysis. This section establishes what changed and what the LLM must generate tests for.
- **Context Section:** Provides execution context surrounding the seed. Includes direct callers (functions that invoke the seed), direct callees (functions the seed invokes), parent/child classes (for inheritance relationships), existing test functions, and execution patterns. This section answers "how is the seed used?" and "what does it depend on?"

- **Instructions Section:** Explicit task directives for test generation. Specifies coverage targets (boundary conditions, happy paths, error cases, edge cases), naming conventions, assertion patterns expected by the repository, and any repository-specific testing idioms.

Hierarchical organisation enables LLMs to prioritise relevant code relationships and infer execution paths without traversing a flat list of nodes. Ego-graph retrieval centred on a seed node; capturing immediate caller/call-ee dependencies and interactions has been shown to be crucial for understanding functional context, with LLM generation conditioned on such subgraph context producing more nuanced, repository-aware outputs [15].

III. EXPERIMENT

A. Test Execution and Validation

To evaluate the capability and robustness of LLM-generated tests, we implement an execution-based framework modelled after the TESTGENEVAL [3] benchmark pipeline. This approach moves beyond static analysis by integrating sandbox execution, dynamic coverage tracking, and mutation testing across three distinct dimensions:

- 1) **Functional Correctness (Pass Rates):** To balance test suite verbosity against strictness, correctness is evaluated using two metrics:
 - *Any Pass@I:* Assesses if at least one generated test in the suite executes successfully, preventing models with large test suites from being heavily penalised for a single faulty method.
 - *All Pass@I:* Evaluates whether every test case in the generated suite passes flawlessly, strictly penalising syntax errors or assertion hallucinations.
- 2) **Structural Code Coverage:** We measure the proportion of source code lines executed by the tests. To isolate valid testing behaviours from broken or failing assertions, this is done strictly via passing tests (*Coverage@pass*).
- 3) **Semantic Fault-Detection (Mutation Testing):** To ensure the tests actively validate logic, we inject synthetic bugs (mutants) into the source code using an automated mutation engine. The test suite is re-executed against each mutant. The mutation score, a measurement of generated test code performance, is calculated as:

$$\text{Mutation Score} = \left(\frac{\text{Mutations Discovered}}{\text{Total Mutants Injected}} \right) \times 100\% \quad (1)$$

Additionally, to measure the efficiency of our approach relative to other test generation methods, we plan to measure the following:

- 1) **Latency of KG Construction:** Wall-clock time to build KG, measured separately for AST parsing and inter-file edge resolution,
- 2) **No. of tokens used by LLM:** Token count of serialised KG sent to LLM, measured with a standard tokenizer.

B. Datasets and Baselines

The evaluation utilises the TESTGENEVAL [3] and SWT-Bench [14] datasets, with sample characteristics summarised in Table II. To ensure robust evaluation, samples are filtered to satisfy four prerequisites: an executable environment, identifiable changed entities, traceable dependency/API context, and the presence of baseline human-written tests.

TABLE II
EVALUATION DATASETS

Dataset	Task	Evaluation use
TESTGENEVAL [3]	Unit test generation and completion	Pass@1, Coverage@pass, Mutation Score
SWT-Bench [14]	Patch-based bug reproduction	Fail-before-fix/pass-after-fix behaviour

C. Initial Works

The implementation is available on GitHub, with the following implemented and validated:

- **Knowledge Graph construction** from Python source files parsed into AST-based nodes and edges.
- **Patch parsing and seed identification** from unified diff patches to map changed code entities to KG seed nodes.
- **Patch-centered subgraph extraction/validation** using bounded BFS with checks for orphaned nodes, broken edges, seed connectivity, and duplicate edges.

D. Future works

The team believes the following future works can be done:

- 1) **Subgraph Size Optimization:** Experimentation with BFS hop counts to determine the ideal knowledge graph size.
- 2) **Comparative Evaluation:** Focus on gathering experimental data to baseline performance against methods in Section A.

IV. CONCLUSION

This paper introduces a patch-aware KG retrieval framework to mitigate LLM hallucinations and address the test oracle problem in repository-level test generation. Using a symbolic $\rightarrow$ neural paradigm, the architecture addresses the weak connection between existing retrieval methods and active code modifications, extracting validated, change-anchored subgraphs for repository-level test synthesis. This deterministic context ensures the LLM receives dependency and behavioural boundaries centred on code deltas.

REFERENCES

- [1] J. Liu, K. Wang, Y. Chen, X. Peng, Z. Chen, L. Zhang, and Y. Lou, "Large language model-based agents for software engineering: A survey," *ACM Transactions on Software Engineering and Methodology*, 2024.
- [2] M. Boukhelif, N. Kharmoum, and M. Hanine, "Llms for intelligent software testing: a comparative study," in *Proceedings of the 7th International Conference on Networking, Intelligent Systems and Security*, 2024, pp. 1–8.
- [3] K. Jain, G. Synnaeve, and B. Rozière, "Testgeneval: A real world unit test generation and test completion benchmark," in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 14 947–14 999.
- [4] X. Deng, J. Da, E. Pan, Y. Y. He, C. Ide, K. Garg, N. Lauffer, A. Park, N. Pasari, C. Rane *et al.*, "Swe-bench pro: Can ai agents solve long-horizon software engineering tasks?" *arXiv preprint arXiv:2509.16941*, 2025.
- [5] S. Haroon, M. T. Khan, and M. A. Gulzar, "Evaluating llm-based test generation under software evolution," *arXiv preprint arXiv:2603.23443*, 2026.
- [6] H. Taherkhani, A. DaghighFarsoodeh, M. Chowdhury, H. V. Pham, and H. Hemmati, "Consistency meets verification: Enhancing test generation quality in large language models without ground-truth solutions," *arXiv preprint arXiv:2602.10522*, 2026.
- [7] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, "The oracle problem in software testing: A survey," *IEEE transactions on software engineering*, vol. 41, no. 5, pp. 507–525, 2014.
- [8] Z. Zhang, C. Wang, Y. Wang, E. Shi, Y. Ma, W. Zhong, J. Chen, M. Mao, and Z. Zheng, "Llm hallucinations in practical code generation: Phenomena, mechanism, and mitigation," *Proceedings of the ACM on Software Engineering*, vol. 2, no. ISSTA, pp. 481–503, 2025.
- [9] H. Wang, Y. L. Gao, Z. Feng, and X. Wang, "Semanticforge: Knowledge graph-enriched code generation for repository-level software development," in *Proceedings of the ACM International Conference on the Foundations of Software Engineering (FSE)*, 2025, pp. 812–825.
- [10] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, and X. Wu, "Unifying large language models and knowledge graphs: A roadmap," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 7, pp. 3580–3599, 2024.
- [11] H. Sansford, N. Richardson, H. P. Maretic, and J. N. Saada, "Grapheval: A knowledge-graph based llm hallucination evaluation framework," *arXiv preprint arXiv:2407.10793*, 2024.
- [12] W. Liu, A. Yu, D. Zan, B. Shen, W. Zhang, H. Zhao, Z. Jin, and Q. Wang, "Graphcoder: Enhancing repository-level code completion via code context graph-based retrieval and language model," *arXiv preprint arXiv:2406.07003*, 2024.
- [13] H. Tao, Y. Zhang, Z. Tang, H. Peng, X. Zhu, B. Liu, Y. Yang, Z. Zhang, Z. Xu, H. Zhang, L. Zhu, R. Wang, H. Yu, J. Li, and P. Di, "Code graph model (cgm): A graph-integrated large language model for repository-level software engineering tasks," 2025. [Online]. Available: <https://arxiv.org/abs/2505.16901>
- [14] N. Mündler, M. N. Müller, J. He, and M. Vechev, "Swt-bench: Testing and validating real-world bug-fixes with code agents," *Advances in Neural Information Processing Systems*, vol. 37, pp. 81 857–81 887, 2024.
- [15] S. Ouyang, W. Yu, K. Ma, Z. Xiao, Z. Zhang, M. Jia, J. Han, H. Zhang, and D. Yu, "Repograph: Enhancing ai software engineering with repository-level code graph," in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 30 098–30 121.
- [16] M. Gardner and T. Mitchell, "Efficient and expressive knowledge base completion using subgraph feature extraction," in *Proceedings of the 2015 conference on empirical methods in natural language processing*, 2015, pp. 1488–1498.